Ray-Based Sampling for Efficient Extraction of Transition Lines in Double Quantum Dot Stability Diagrams

Ehsan Alizadeh, Takafumi Fujita, Akira Oiwa

SANKEN , Osaka University

SANKEN 大阪大学 産業科学研究所

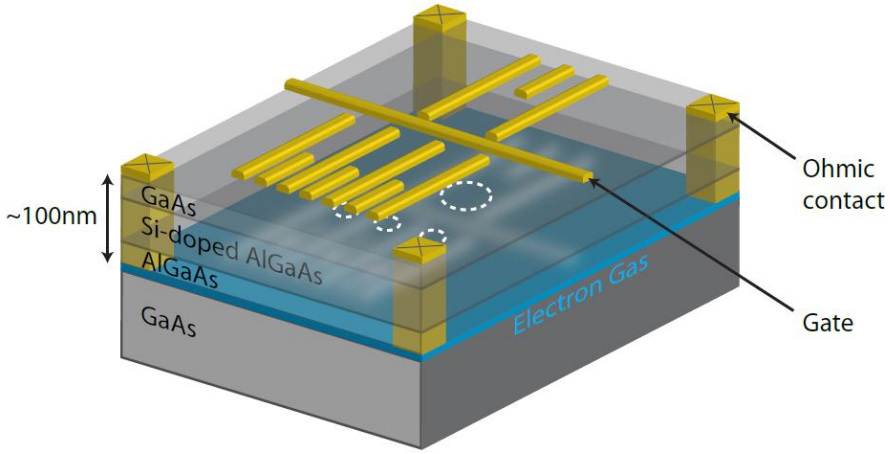
大阪大学 OSAKA UNIVERSITY

Outline

- Introduction
- Previous work
- Ray-based classification
- Device simulation with QArray
- Ray-based measurement
- Dataset and neural network
- Results
- Summary

Introduction

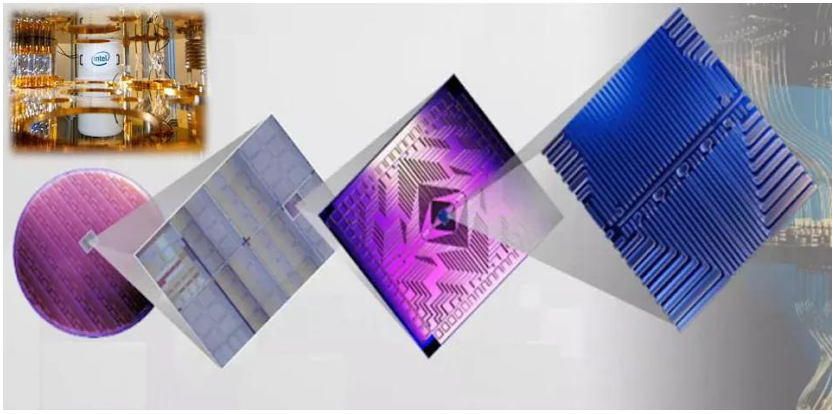
Gate-Defined Quantum dot array



High-fidelity qubit operations

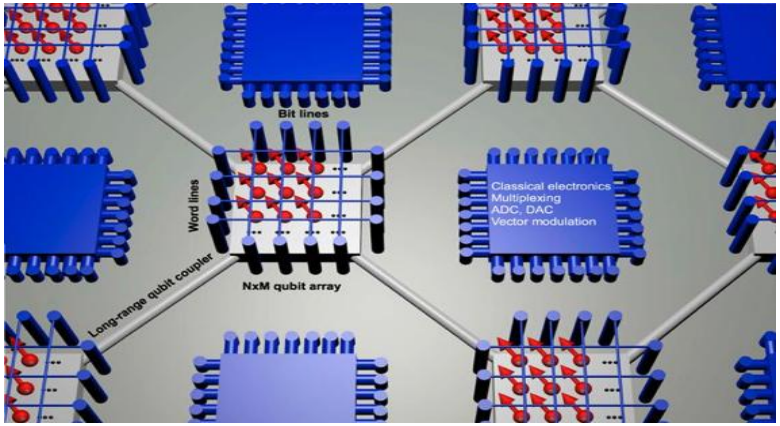
- ✓ Single qubit operation
- ✓ Two qubit operation

Compatible with industrial CMOS technology



Intel's Tunnel Falls Quantum processor

Scalable architecture

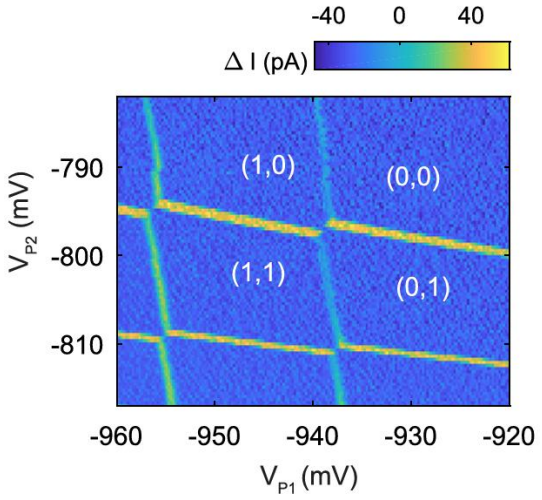


L. M. K. Vandersypen et al. npj (2018)

The problem: measuring the charge stability diagram

- Every quantum-dot device must be characterised through its charge stability diagram before any qubit operation.
- That means a full raster scan of the (V_1, V_2) plane — slow, and repeated for every gate pair, on every device.
- But almost none of that map is what we actually need.

In a conventional raster scan of a double-quantum-dot stability diagram, the charge transition lines that carry the device parameters occupy 3–5% of the measured pixels. The remainder of the acquisition budget is spent on Coulomb-blockaded regions of nearly constant sensor response.



Charge transition lines carry everything we need:

- The charge configuration of each dot
- The interdot coupling
- The virtual-gate transformation

Question : Can we measure a few percent of the plane and still recover every transition line?

W. I. L. Lawrie et al., Appl. Phys. Lett. 116, 080501 (2020)

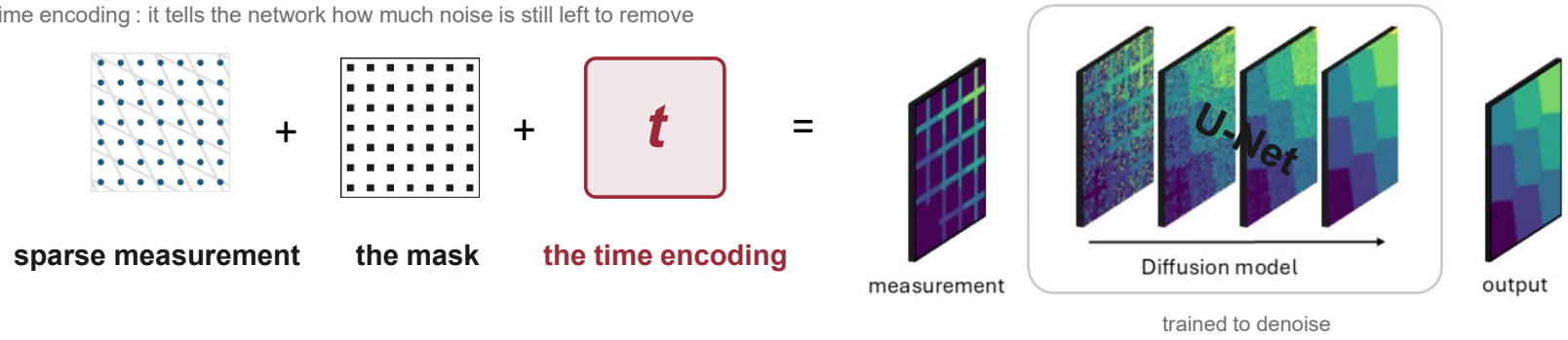
Previous work

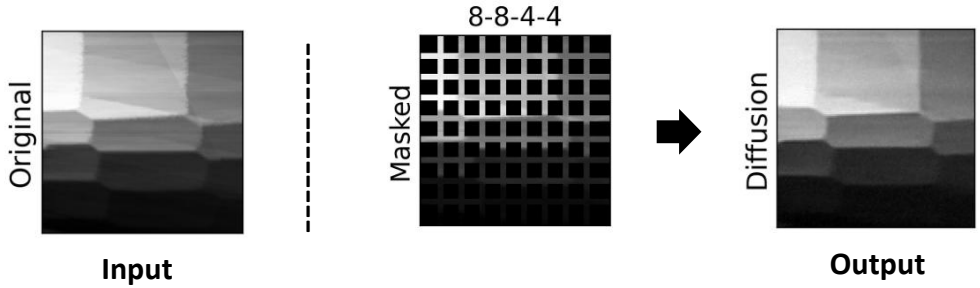
Reconstructing CSDs with diffusion models

V. Hernandes et al., 2026

- A diffusion model fills in the full CSD from a sparse mask — a uniform grid, or straight line cuts.
- ~4 % of the data is enough.
- Learned from 9,850 measured diagrams (real Si/SiGe and Ge/SiGe devices, 128 × 128 pixels)
- Each iteration: take a clean CSD, add noise per the diffusion schedule
- Model predicts the denoised image from the noisy image + sparse measurements + mask + timestep
- Loss: MSE between original and prediction

Time encoding : it tells the network how much noise is still left to remove

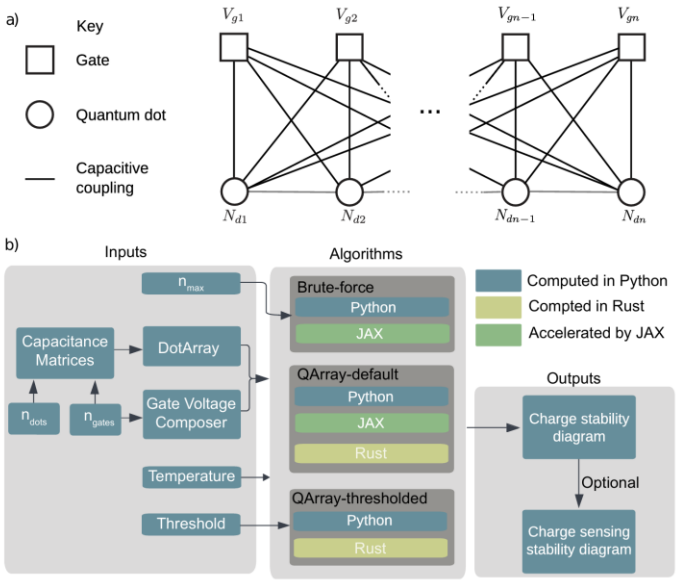


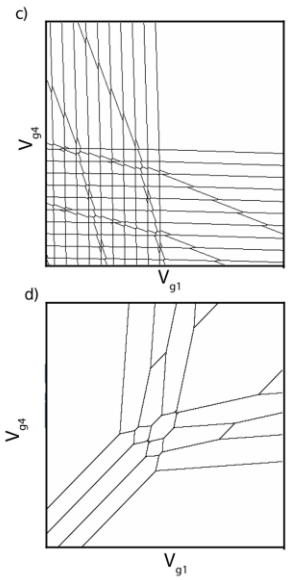


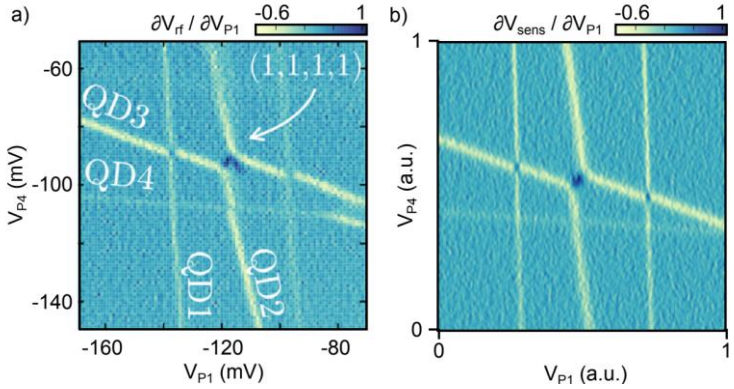
This work: sparse ray measurement → U-Net → probability of a transition line at every pixel

Device simulation: QArray

- QArray is a fast constant-capacitance simulator for quantum-dot arrays (van Straaten et al., 2024) — fast enough to act as a DIGITAL TWIN of a real quantum dot device. (100 ×100 pixels charge stability diagram of a 16-dot array is simulated in less than a second)



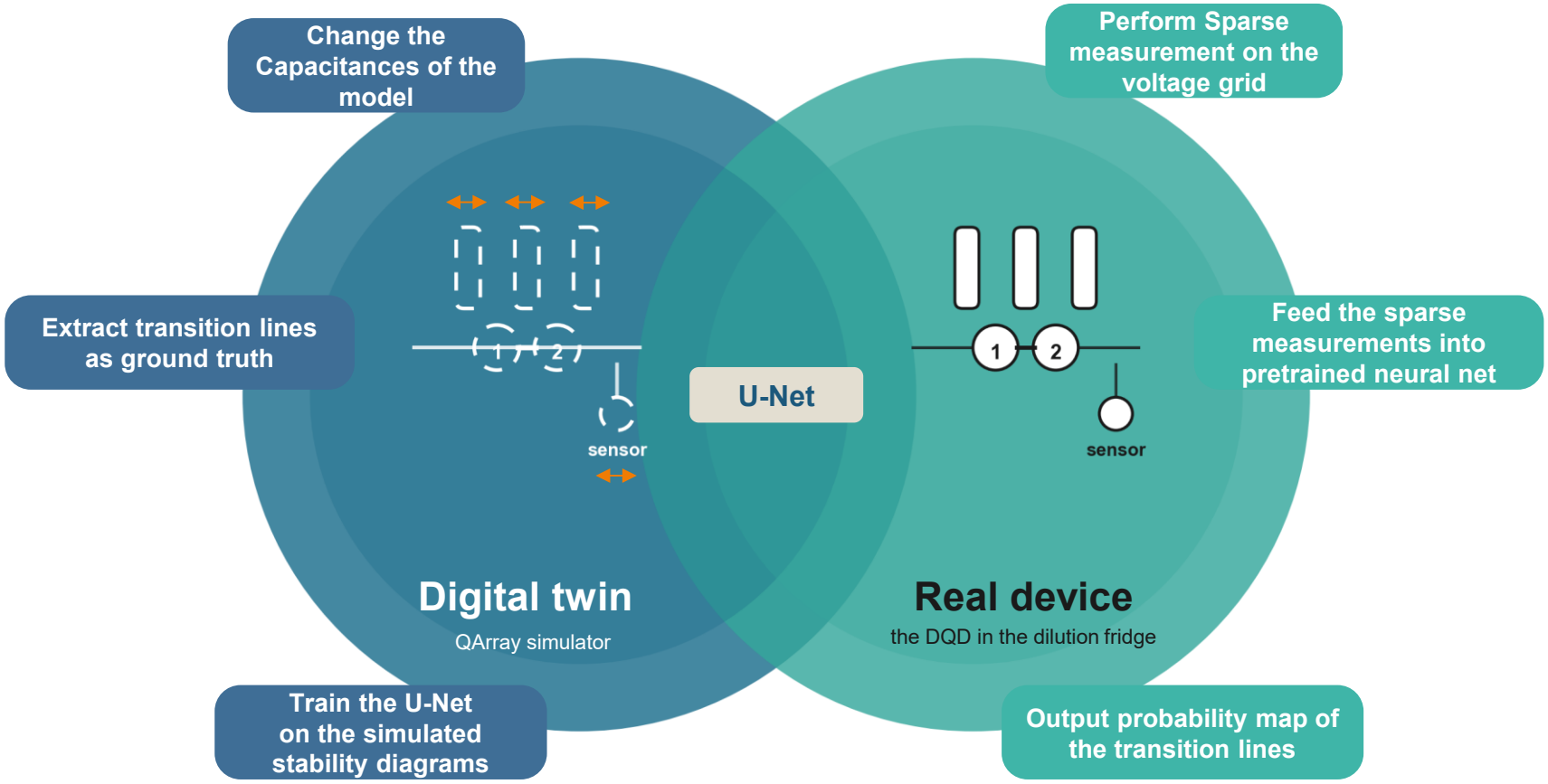




(a) a MEASURED stability diagram of a linear quadruple dot array against the two outermost plunger gates V_P1 and V_P4; colour is dV_rf / dV_P1.
(b) the same diagram recreated in QArray.
adapted from van Straaten et al., QArray (2024)

(a) Generic quantum dot array schematic.
(b) Qarray package structure (adapted from van Straaten et al., 2024).
(c) Charge stability diagrams: hole-based 4-dot linear array (open regime).
(d) Computed charge stability diagrams: 4-dot linear array (closed regime, 4 charges), via Rust algorithm.

QArray as a digital twin builder: training the transition-line probabilistic model



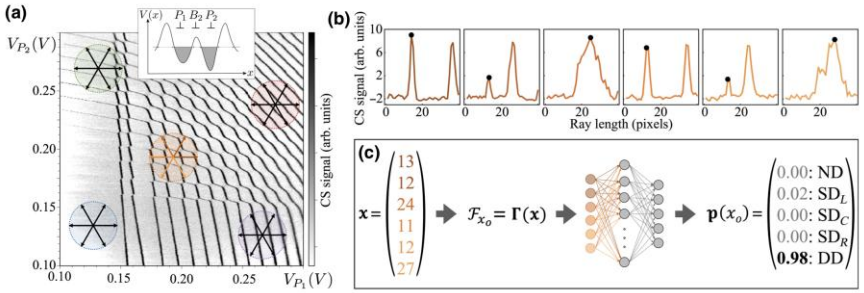
QArray is the digital twin builder: It simulates double dots with randomized capacitances (C_{dd}, C_{gd}, C_{ds}, C_{gs}), giving exact transition lines by construction.

Sparse measurements of the plunger-gate plane of charge sensor are the input, transition lines are the target.

Ray-based classification (RBC)

Rays to classify a STATE

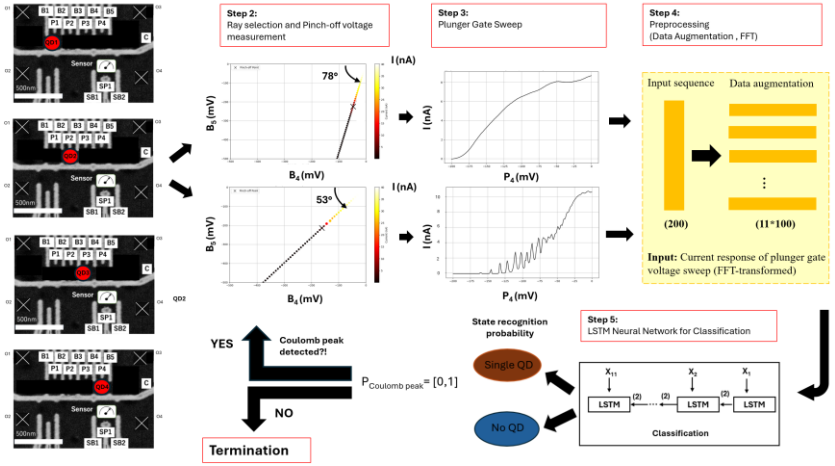
J. P. Zwolak et al., PRX Quantum 2, 020335 (2021)



- Rays are fired from one point in gate space; the peak positions along them become a feature vector.
- A network maps that vector to P(state): no dot, single dot, double dot.

Our earlier RBC pipeline for automated tuning

E. Alizadeh Kashtiban et al., JJAP 64, 02SP22 (2025)



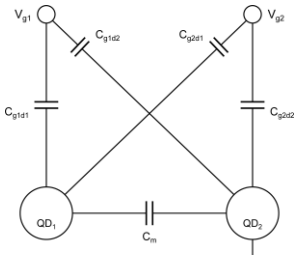
- Device : GaAs/AlGaAs QD device
- A ray at a random angle in barrier-gate space finds the pinch-off; Then we perform plunger gate sweep.
- An LSTM reads each trace for Coulomb peaks — peaks mean a single dot has formed.

The whole idea: characterise the two-dimensional voltage space with one-dimensional rays.

From simulator to a probabilistic model

QArray supplies the training dataset for simulation; the rays supply a few percent of the data; U-net joins them .

TRAINING DATASET



$$\mathbf{C_{dd}} = \begin{bmatrix} c_{d_1d_1} & c_{d_1d_2} \\ c_{d_2d_1} & c_{d_2d_2} \end{bmatrix}$$

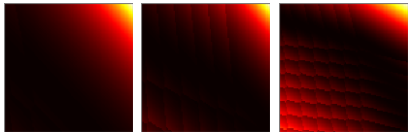
$$\mathbf{C_{dg}} = \begin{bmatrix} c_{d_1d_1} & c_{d_1d_2} \\ c_{d_2d_1} & c_{d_2d_2} \end{bmatrix}$$

$$\mathbf{C_{sd}} = \begin{bmatrix} c_{s1d1} & c_{s1d2} \end{bmatrix}$$

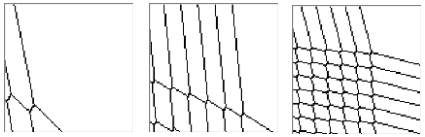
$$\mathbf{C_{sg}} = \begin{bmatrix} c_{s1g1} & c_{s1g2} & c_{s1g3} \end{bmatrix}$$

the device model behind the training dataset

Charge sensor data

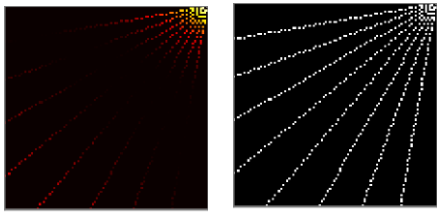


Ground truth data



550 simulated devices —
Top: the charge sensor the ray's sample.
Bottom: the exact lines, the truth label.

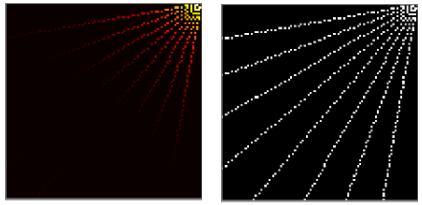
TRAINING PHASE



ch 1 sparse signal

ch 2 sampling mask

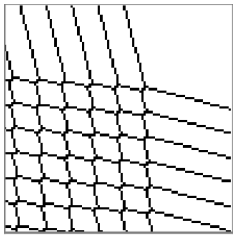
TEST PHASE



ch 1 sparse signal

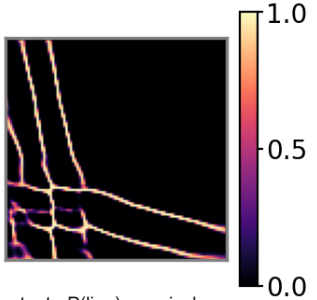
ch 2 sampling mask

U-Net



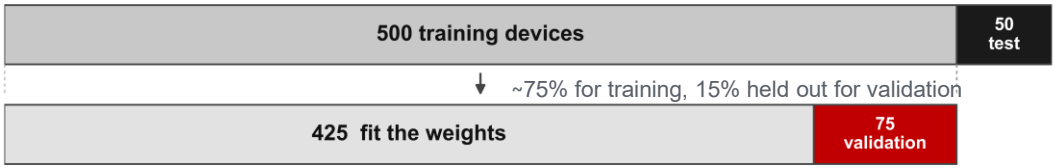
target: the transition lines

The same U-Net



output: P(line) per pixel

Training dataset vs Test dataset : 550 simulated devices

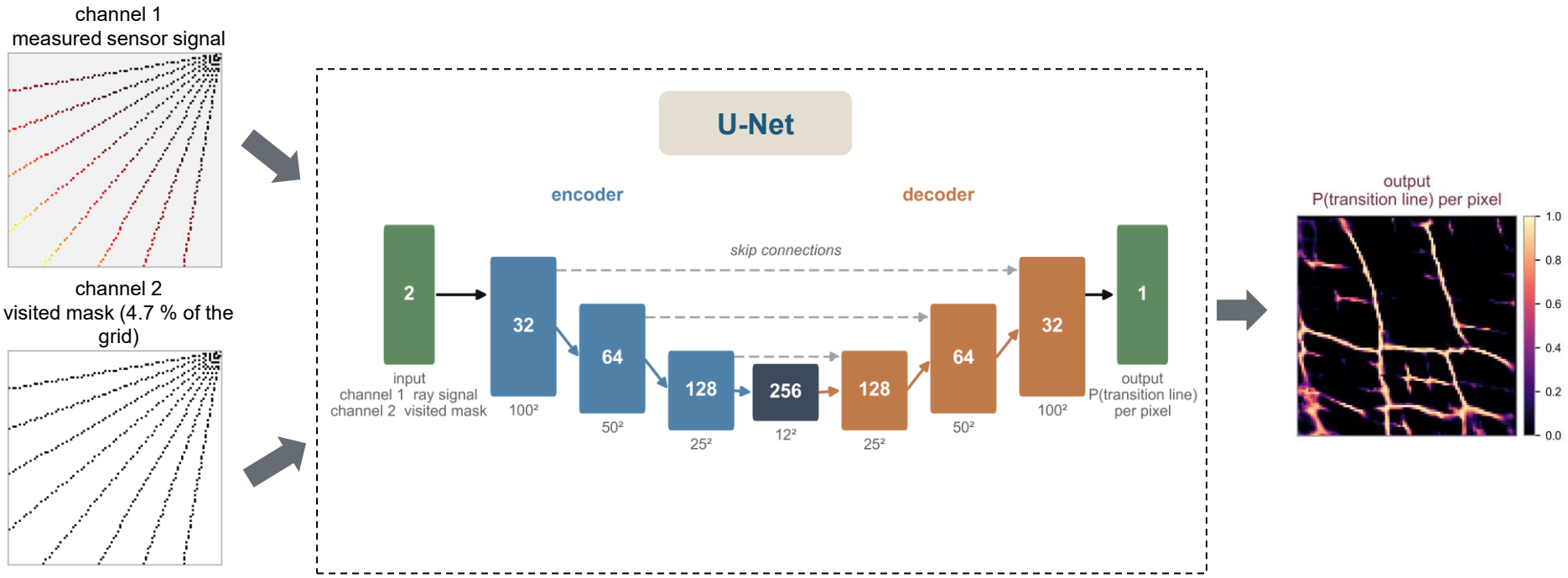


Note: In this work ,U-net is verified on the 50 held-out simulated devices only. Verification on real rxperimental diagrams are future work.

Predicting the lines, not the whole diagram

Reconstructing the whole stability diagram is the wrong target: transition lines are only ~7 % of the pixels, so “no transition line anywhere” is already 93 % correct.

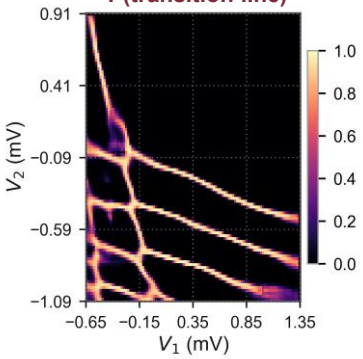
-> Missing a transition line is penalised far more heavily than labelling a Coulomb-blockade region as a line (weighted BCE)

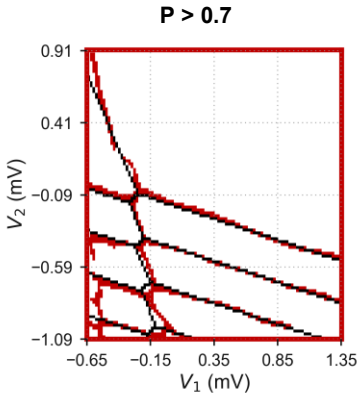


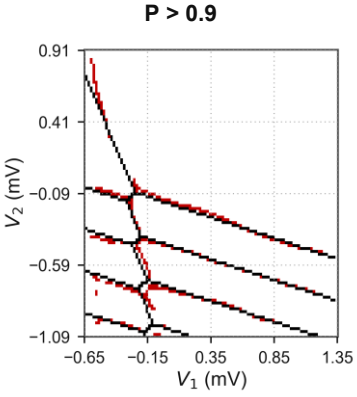
- Two channels in: the charge sensor value of where a ray passed, and a binary mask of where that measurement was.
- Encoder — zooms out: less detail, more of the picture at once, so scattered ray hits read as one line.
- Decoder — zooms back in: full size again, re-using the detail saved on the way down.
- Head — decides: one number per pixel, 0 to 1 — a probability, not a yes/no.

From probability-map to transition lines

Two tasks ahead. ① BINARISE the output — the probability map is continuous, transition lines are binary, so every pixel must become 0 or 1. Where we cut is the THRESHOLD.

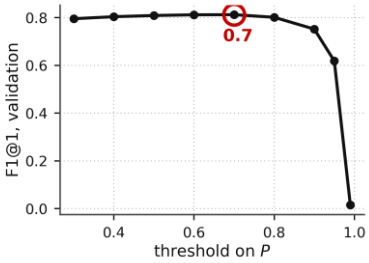




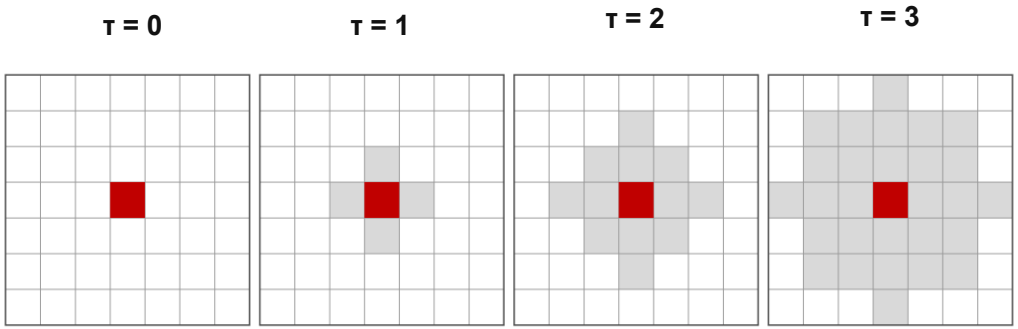


① How to find Threshold value?!

How: sweep the cut 0.3 → 0.9, score F1 on the 75 VALIDATION devices, keep the best (here P=0.7).
And use it when applying model to test dataset.



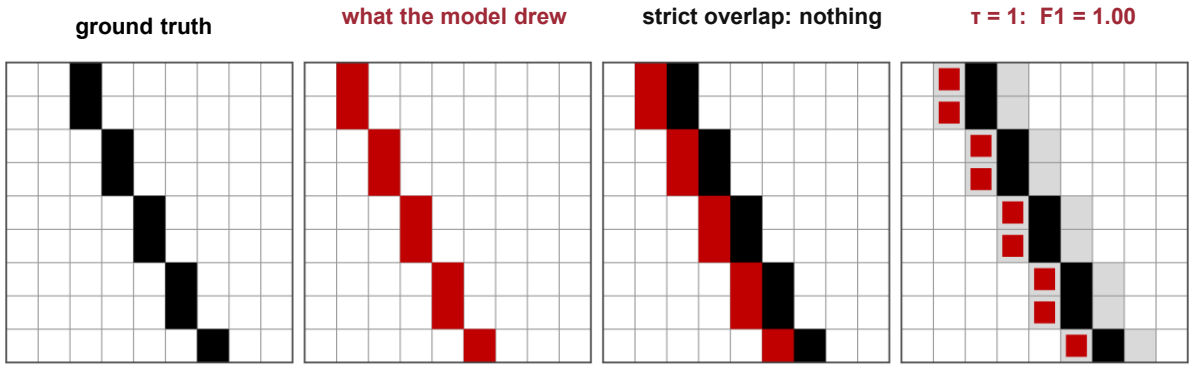
② EVALUATE the output — comparing it with the truth pixel for pixel is brutal on lines one pixel wide, so a drawn pixel may be a little off: that slack is the TOLERANCE τ.

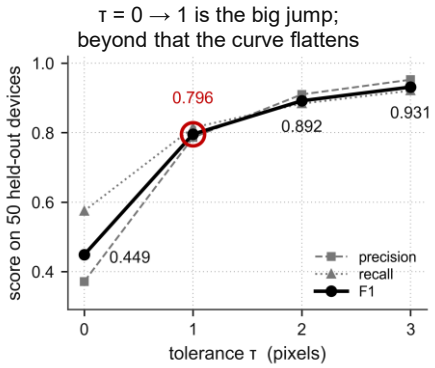


The model's output never changes — τ only changes how strictly we read it. The red pixel is what the model drew; it still counts as correct when a true line passes anywhere in its grey patch.
τ = 0 demands an exact hit. τ = 1 adds the four side neighbours. A diagonal is 1.41 pixels away, so it only counts from τ = 2.

Tolerance τ

The same predicted transition line — same shape, same slope — displaced by one pixel across its own direction. Not a single pixel coincides, so strict pixel F1 = 0.00. The evaluation therefore has to be loosened: we report F1@τ, where a predicted pixel counts if a true line pixel lies within τ pixels of it, and vice versa.



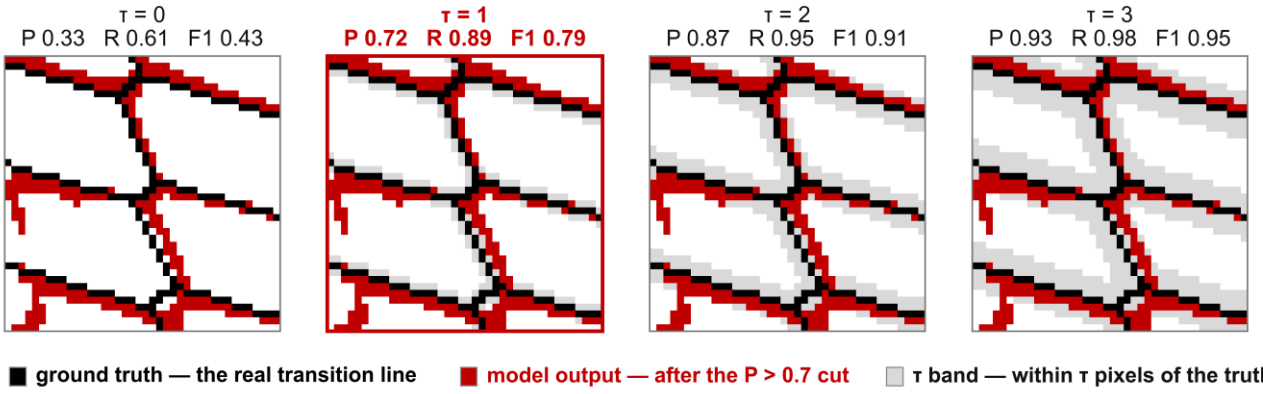


precision@τ — of the pixels drawn, how many lie near a true line? (band around the TRUTH).

recall@τ — of the true lines, how many were covered? (band around the PREDICTION).

F1@τ is the harmonic mean; it increases with τ.

Same ground truth, same U-net output — only the tolerance band grows

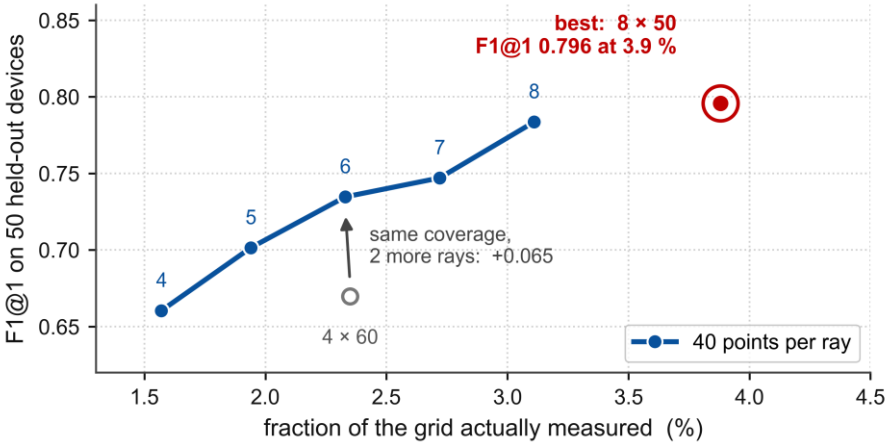


Results

What we learn: trade off between accuracy vs grid coverage
— **3.9 % of the plane already gives F1@1 = 0.796**.

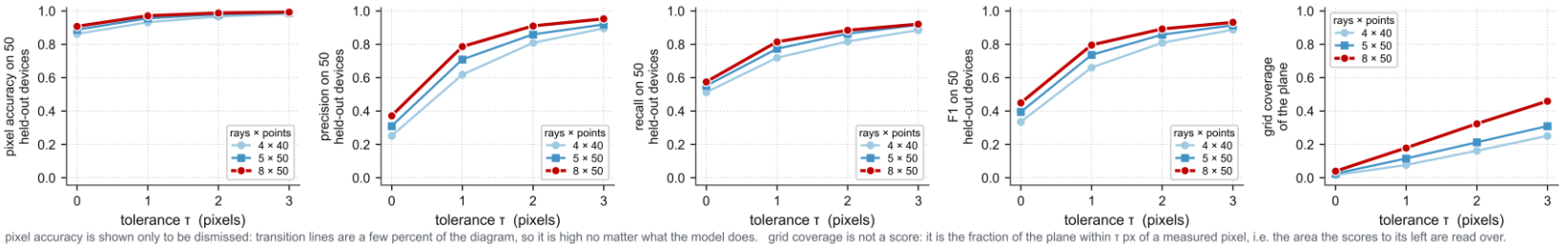
rays × points	coverage	precision	recall	accuracy	F1@1
4 × 40	1.6 %	0.617	0.719	93.2 %	0.660
6 × 40	2.3 %	0.683	0.804	95.3 %	0.735
8 × 40	3.1 %	0.748	0.830	96.4 %	0.783
8 × 50	**3.9 %**	**0.786**	**0.815**	**97.1 %**	**0.796**

50 held-out devices.
F1@1: a predicted pixel counts when a true line lies within 1 px.



the number beside each point is the ray count.
4 of the 15 budgets did not converge and are not shown.

- 3.9 % of the plane, 25× fewer points: 81 % of the true line pixels found, 79 % of drawn ones real.
- τ = how many pixels a line may be off: 0 → 1 gains 0.35 in F1, 1 → 3 only 0.14 — **so τ = 1 is enough**.



pixel accuracy is shown only to be dismissed: transition lines are a few percent of the diagram, so it is high no matter what the model does. grid coverage is not a score: it is the fraction of the plane within τ px of a measured pixel, i.e. the area the scores to its left are read over.

Summary and future prospects

Summary

- A U-Net recovers the transition lines of a DQD from rays covering a few percent of the gate-voltage plane, and returns a probability at every pixel rather than a binary mask.
- F1@1 = 0.849 from 4.7 % of the plane — about 21× fewer measured points than a raster scan.
- The threshold is chosen on a validation split carved from the training devices

Future prospects

- Physics-informed learning — impose the constant-capacitance model —as a loss term associated with it.
- Reinforcement learning — let an agent choose where to fire the next ray, aiming at the pixels where the probability of finding transition lines is the highest.
- Experimental data — everything here is SIMULATED. The next step is to train and test on real experimental stability diagrams.

Ehsan Alizadeh (Oiwa lab) : u131475d@alumni.osaka-u.ac.jp